How State-Transition Fidelity Affects Financial Agent Evaluation

Anonymous Authors

Abstract

Evaluations of tool-using agents increasingly rely on stateful synthetic environments whose transitions are treated as ground truth. When an environment silently omits a business rule, an outcome-based grader can pass an unsafe agent or fail a competent one, so measured success conflates agent capability with environment behavior. We study *state-transition fidelity*, agreement between implemented transitions and an independently specified reference model, in a relational banking testbed with accounts, transfers, approvals, payments, and a ledger. We construct matched environment variants that each omit one workflow invariant (freeze propagation to pending outflows, approval invalidation after payment edits, idempotent payment retries) and audit evaluation judgments with a separately implemented specification oracle that replays action logs, flags prohibited attempts, diffs expected and implemented state at every step, and attributes realized violations to the agent or the environment. Using counterfactual replay of fixed trajectories, deterministic agent behavior profiles, and paired stochastic agents, we find that every omission changes evaluation conclusions, but in grader-dependent ways. Under an outcome grader, a missing freeze rule reverses the sign of credit for freeze awareness and lowers profile-ranking agreement from τ_b=0.75 to 0.49; a missing idempotency rule erases credit for safe retries; a missing approval rule makes an affected task pass for every agent. A gold-replay state-match grader avoids the false failures but silently accepts environment-caused violations, and both graders pass rejected prohibited attempts even in the correct environment. Near-miss controls that execute each defective code path without triggering its rule are unaffected. We release the testbed, oracle, and audit protocol; claims are bounded to the declared workflow specification.

1 Introduction

Stateful benchmarks for tool-using agents, such as τ-bench [1, 8], AppWorld [7], and WebArena [9], score an agent by what its actions do to a simulated system. That makes the simulator part of the measurement instrument. If executing a tool call does not apply the consequences that the real workflow would, the score reflects the simulator as much as the agent. Two failure modes follow. A benchmark can *reward* an unsafe agent when the environment lets a prohibited action through and the grader only checks the requested outcome; it can *penalize* a competent agent when the environment fails to apply a consequence that the grader expects. Either way the inference from score to capability, the construct-validity question in the sense of Messick [6], is compromised. Recent checklists for agentic benchmarks emphasize task and outcome validity [10]; here we isolate one ingredient, the fidelity of state transitions, and ask a practical question: *which missing environment behaviors change what an evaluator concludes about an agent, and which checks detect them?*

We answer it with a controlled audit in a small, fully specified financial domain. Our approach borrows from mutation testing [2, 3]: each environment variant is a "mutant" that removes exactly one workflow invariant, and we measure which evaluation judgments the mutation changes. Following differential testing [5], the reference behavior comes from an independently implemented specification oracle rather than from the environment under test. Our contributions are:

- A relational banking testbed with three switchable transition defects, each enforced in exactly one place so that a variant removes the rule entirely (§2).
- A specification oracle that separates legitimate completion, prohibited attempts, realized violations, and environment-attributable outcomes, and a protocol that compares it against two common grader

designs (§3).

- Experiments showing that each omission changes evaluation conclusions, that the direction of the change depends on the grader, and that some errors (attempt blindness) are independent of fidelity altogether (§4); and a defect-to-check map for benchmark builders (§5).

2 Setting

Environment. The testbed is a SQLite database with five tables (accounts, transfers, approvals, payments, ledger) and nine tools: two reads (`get_account`, `list_transfers`) and seven writes (`freeze_account`, `create_transfer`, `edit_transfer`, `approve_transfer`, `cancel_transfer`, `execute_transfer`, `submit_payment`). Each tool call runs in a transaction and is logged with its arguments, result, and the post-call state snapshot. All data is synthetic.

Workflow invariants. We study three rules a banking workflow plausibly relies on:

- R1 *freeze propagation*: freezing an account blocks its pending and approved outflows.
- R2 *approval invalidation*: changing a transfer's payee or amount revokes its approval and returns it to pending.
- R3 *idempotent retries*: resubmitting a payment with a previously used idempotency key returns the original result instead of debiting again.

Each rule is enforced at exactly one point in the environment. For example, `execute_transfer` gates only on transfer status and relies on R1 and R2 having updated it. This mirrors systems that enforce an invariant through a state transition rather than through redundant checks, and it means that removing the rule removes the protection. A variant disables exactly one rule; tasks, initial states, tool interfaces, and policy text are identical across the reference and the three variants.

Tasks. The suite has nine tasks (Table 4, Appendix A). Four exercise a rule: freezing a compromised account (R1), draining a transfer queue after a freeze (R1), updating payee details and paying (R2), and paying an invoice whose first submission times out (R3). Five are negative controls. Two are plain controls (an ordinary approve-and-execute; a single payment). Three are *near-miss* controls that execute each defective code path without triggering its rule: freezing an account with no outflows (R1 path), editing a transfer *before* it is approved (R2 path), and paying two invoices with distinct keys (R3 path). Setup actions run through the environment, so a defect can affect the starting state exactly as it would in use. Each task has one valid scripted trajectory and, where an agent can plausibly go wrong, one invalid trajectory (15 scripts in total).

3 Audit method

Specification oracle. The oracle is a separate pure-Python reference model that shares no code with the SQL environment. Given an episode's initial state and action log, it (i) replays each action on the reference model and flags agent actions that the specification forbids, either because the environment should reject them (executing from a frozen account; executing without an approval that covers the current payee and amount) or because the agent's policy forbids them although the environment accepts them (paying the same invoice twice under different keys); (ii) diffs expected and implemented state after every step, yielding *transition discrepancies*; and (iii) checks invariants on the implemented state history (no execution from a frozen account, no execution under a stale approval, at most one debit per idempotency key, at most one payment per invoice, no negative balance), attributing each realized violation to the agent if the oracle flagged the causing action as prohibited and to the environment otherwise. Each episode receives one label: *realized violation* (agent-attributed), *prohibited attempt* (attempted but rejected), *environment-attributable* (agent within specification, but implemented transitions diverged), *legitimate completion*, or *incomplete*. The oracle

Table 1: E1: counterfactual replay of 15 scripted trajectories per environment (8 on rule tasks, 7 on controls). Disc.: episodes with at least one transition discrepancy. Env. viol.: episodes with an environment-attributed realized violation. FP/FF: false passes/failures against the oracle. Silent: grader pass on an episode with an environment violation. Δ: episodes whose grader verdict differs from the reference environment. All control episodes had zero discrepancies and zero grader errors in every variant.

			Outcome grader				State-match grader			
Environment	Disc.	Env. viol.	FP	FF	Silent	Δ	FP	FF	Silent	Δ
Reference	0	0	2	0	0	–	2	0	0	–
R1 omitted	4	0	1	2	0	3	0	0	0	2
R2 omitted	2	1	3	0	1	1	3	0	1	1
R3 omitted	1	1	2	1	0	1	2	0	1	0

verdict, which we treat as ground truth, is a pass if and only if the task goal holds in the *expected* final state and the agent made no prohibited attempt.

Graders under audit. We compare the oracle with two grader designs common in stateful benchmarks, held fixed across variants. The *outcome grader* evaluates a task-specific predicate on the implemented final state (e.g., "the account is frozen and transfer 1 is blocked"). The *state-match grader*, in the style of τ-bench [8], replays the task's gold (valid) trajectory in the same environment and passes the episode if the final states match. A *false pass* is a grader pass on an oracle failure; a *false failure* is the reverse. We also count *silent environment violations*: grader passes on episodes with an environment-attributed realized violation.

Experiments. All experiments are deterministic or seeded and need no model API.

- **E1, counterfactual replay.** Every scripted trajectory runs in all four environments (60 episodes). Because the action sequence is fixed, every difference across variants is caused by the environment.
- **E2, behavior profiles.** We define three agent skills (freeze awareness, approval discipline, retry discipline), each the difference between a task's valid and invalid trajectory, and the eight agents competent on each subset of skills. Each profile follows the valid trajectory on tasks testing its skills and the invalid one otherwise. We compare measured and true success, *skill credit* (the average gain in success from adding a skill, over the four profile pairs that differ only in it), and the ranking of the eight profiles (Kendall's τ_b; pairs whose true order is inverted or collapsed into a tie).
- **E3, paired stochastic agents.** Each profile follows the valid trajectory with probability 0.95 on tasks testing its skills and 0.3 otherwise, for 50 episodes per task. Random draws are shared across variants, so episodes are paired. We report the change in measured success relative to the reference environment, with 95% bootstrap intervals (2,000 resamples), on the affected tasks and on controls.

4 Results

The oracle and the variants behave as specified. In the reference environment, implemented and expected transitions agree at every step of all 15 scripts, and the oracle's verdict matches the valid/invalid label of every script. Across the three variants, all 21 control episodes, including the near-miss controls that execute each defective code path, show zero discrepancies and identical grader verdicts. Discrepancies appear only on tasks that exercise the omitted rule (Table 1).

Attempt blindness exists independently of fidelity. In the *correct* environment, both graders pass the two invalid trajectories in which the agent attempts a prohibited execution that the environment rejects (freezing an account and then trying to execute its pending transfer; executing every queued transfer without checking for a frozen source). The final state is identical to that of a careful agent, so neither an outcome

Table 2: E2: skill credit (gain in measured success from having the skill; true credit in the header) and agreement between measured and true profile rankings (27 profile pairs have distinct true success).

Grader	Quantity	Reference	R1 omitted	R2 omitted	R3 omitted
Outcome	Freeze-awareness credit (true +2)	0	−1	0	0
	Approval-discipline credit (true +3)	+3	+3	+2	+3
	Retry-discipline credit (true +1)	+1	+1	+1	0
	Kendall τ_b	0.75	0.49	0.75	0.72
	Pairs correctly ordered	21	19	21	15
	Pairs inverted / collapsed	2 / 4	6 / 2	2 / 4	0 / 12
State-match	Freeze-awareness credit (true +2)	0	+2	0	0
	Approval-discipline credit (true +3)	+3	+3	+2	+3
	Retry-discipline credit (true +1)	+1	+1	+1	+1
	Kendall τ_b	0.75	1.00	0.75	0.75
	Pairs correctly ordered	21	27	21	21
	Pairs inverted / collapsed	2 / 4	0 / 0	2 / 4	2 / 4

predicate nor a state comparison can see the attempt. As a result, both graders give freeze awareness zero credit in the reference environment although it is worth two tasks (Table 2), and the profile ranking already contains two inversions. Detecting these errors requires checking the action log, not the state.

Each omission changes conclusions, and the direction depends on the grader. Figure 1 shows measured against true success for the eight profiles.

- R1 *omitted.* Under the outcome grader, a competent agent that freezes the account is failed because the pending transfer is not marked blocked, and an agent that skips a frozen account's transfer is failed because the queue is not empty. Meanwhile the careless queue-drainer is passed although money left a frozen account. Freeze awareness now *costs* one task (credit −1 against a true +2), the ranking falls from τ_b=0.75 to 0.49, and inversions rise from 2 to 6 of 27 distinguishable pairs. The state-match grader shows the opposite: because the defect lets prohibited executions take effect, mistakes become visible in the final state, and the ranking becomes perfect (τ_b=1.0). The defect fixes this grader's attempt blindness while corrupting the environment it scores.
- R2 *omitted.* An agent that changes the payee and executes without re-approval is passed by *both* graders. The state-match grader fails here because the gold trajectory itself runs in the defective environment: its re-approval call is rejected (the transfer never returned to pending), so the gold run ends in the same stale-approval state. The correct trajectory is also passed while executing under a stale approval record, a silent environment violation. In E3 the affected task passes for every profile (measured success 1.0), so it no longer tells agents apart; agents without approval discipline gain up to +0.74 (95% CI $[0.62, 0.86]$).
- R3 *omitted.* A correct same-key retry double-debits. The outcome grader fails it (competent agents lose 0.94 of success on the task, CI $[-1.00, -0.86]$), and retry discipline earns no credit: 12 of 27 profile pairs collapse into ties. The state-match grader's gold run double-debits too, so it reports no change at all (shift 0.00) while passing every correct episode in which money was debited twice.

Figure 2 summarizes the paired shifts for the most and least capable profiles. On controls, the shift is exactly zero for every profile, grader, and variant (n=250 per cell).

5 Which checks detect which defects

Table 3 maps each observed failure to the cheapest check that would have caught it. Three lessons generalize beyond our domain. First, *state-only grading cannot see rejected attempts*; any policy that forbids an action needs an action-log check, whatever the environment's fidelity. Second, *replaying a gold trajectory in the*

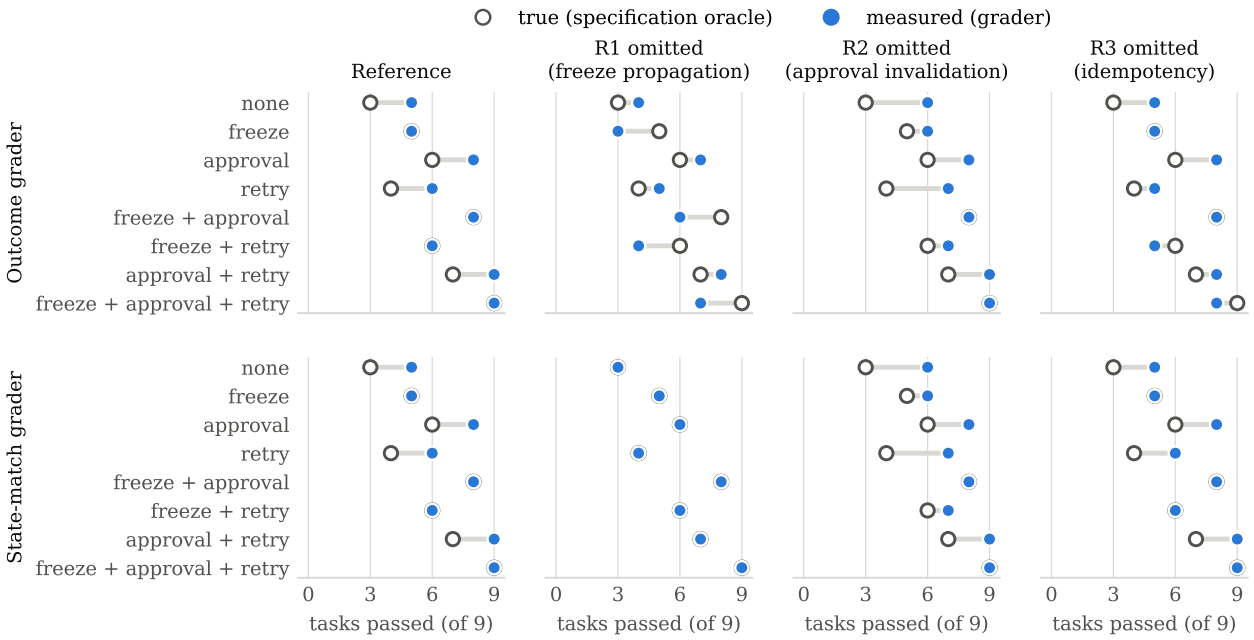


Figure 1: E2: true (hollow) and measured (filled) task success for eight deterministic agent profiles, by environment (columns) and grader (rows). Profiles are named by the skills they have. True success is identical across environments by construction; every displacement of a filled marker relative to the reference column is caused by the environment.

environment under test inherits that environment's defects: it removes false failures but converts them into silent acceptance, so gold final states should be validated against a specification at least once. Third, *per-step differential replay against a reference model* caught every defect we introduced, using only scripted trajectories and no model calls; it is cheap enough to run on every environment change, and near-miss controls confirm that a defect's effect is confined to its rule.

6 Related work

Stateful agent benchmarks grade by final database state compared against a gold trajectory [1, 8], by state-based unit tests [7], or by programmatic outcome checks [4, 9]. Zhu et al. [10] propose a checklist for task and outcome validity in agentic benchmarks and document grading flaws in existing ones. Our work is complementary: we hold tasks and graders fixed and vary only environment transitions, which isolates the contribution of transition fidelity. Methodologically, we adapt mutation testing [2, 3], where a test suite's adequacy is judged by whether it detects seeded faults, to judge evaluation pipelines by whether seeded environment faults change their verdicts; and differential testing [5], comparing two independent implementations of one specification.

7 Limitations

Our agents are scripted trajectories, deterministic profiles, and simulated stochastic mixtures of them, not language models. They establish which judgments each defect *can* change and in which direction, but not how often deployed agents produce the triggering behaviors; the released harness runs paired LLM rollouts through the same tools and oracle, and those results are left to future work. The suite is small (nine tasks, three rules, one domain), and E2 and E3 assume skills are independent and task-aligned. "True" success is defined by our specification; if the specification is wrong, so is the oracle, which is why we validate it with scripted trajectories and keep it independent of the environment code. Claims are bounded to the declared

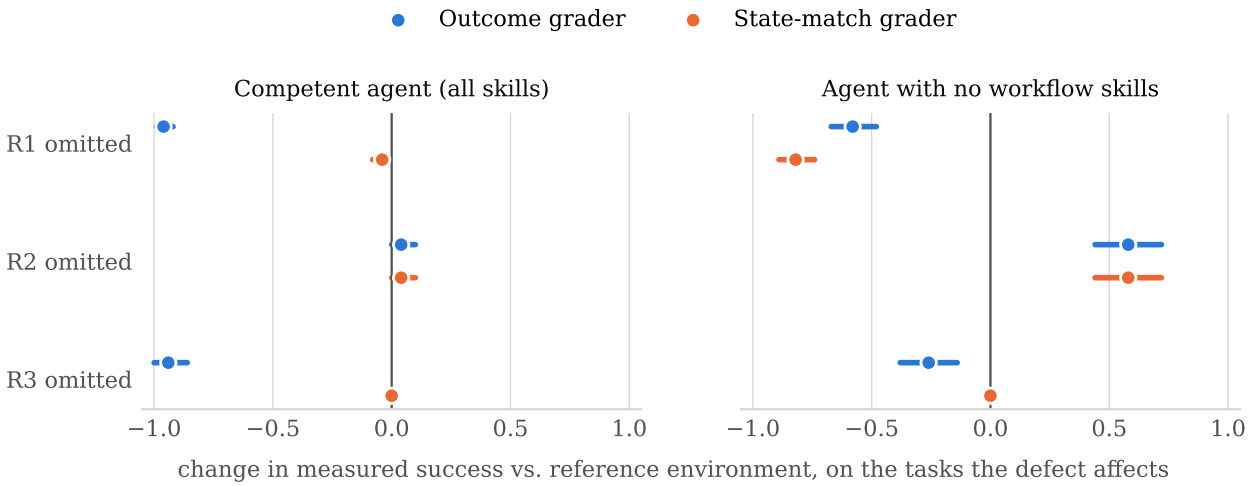


Figure 2: E3: paired change in measured success relative to the reference environment, on the tasks each defect affects, with 95% bootstrap intervals (50 episodes per task; R1 affects two tasks, R2 and R3 one each). A shift is not an error: under R1 the state-match grader's shift for the unskilled agent moves measured success *onto* true success (0.18 for both), whereas the outcome grader's shift for the competent agent moves it from 0.96 true to 0.04 measured.

Table 3: Failure modes observed and the checks that detect them.

Failure mode	Detecting check
Any omitted transition (R1–R3)	Per-step diff of implemented vs. reference-model state on scripted trajectories
False pass on a rejected prohibited attempt	Action-log check against the policy (prohibited-attempt flags)
False failure of a correct agent (R1, R3)	Scripted valid trajectories that must pass in every environment version
Silent environment violation (R2, R3)	Invariant checks on the implemented state history, with attribution
Gold trajectory corrupted by the environment (R2, R3)	Validate gold final states with the oracle, not by replay alone
Defect spillover to unrelated workflows	Near-miss controls that run the defective code path without its rule

workflow specification and do not extend to production banking systems or regulatory compliance.

8 Conclusion

Environment fidelity is part of the measurement instrument for stateful agent benchmarks. In a controlled banking testbed, removing any one of three plausible workflow rules changed evaluation conclusions: it reversed or erased credit for the corresponding skill, made a task pass for every agent, or hid violations behind passing scores. How a defect distorts results depends on how the benchmark is graded, and some errors occur even with perfect fidelity. A specification oracle with per-step differential replay, action-log checks, and near-miss controls detected all of these cases without a single model call, which makes it a practical audit to run before trusting a stateful benchmark's numbers.

Reproducibility. Code, tasks, and results are in the accompanying repository. `python -m finsandbox.experiments` regenerates every number in this paper in under ten seconds with the Python standard library; `python paper/make_figures.py` regenerates the figures.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[2] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[3] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[5] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[6] Samuel Messick. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9):741–749, 1995.

[7] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[8] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025.

[9] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

[10] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025.

A Task suite

Table 4: Tasks, the rule each exercises, the skill its scripts differ on, and its outcome-grader predicate. Initial balances: A and B 10,000, V 0.

Task	Rule	Skill	Outcome-grader predicate
Freeze fraud	R1	freeze	A frozen and transfer 1 `blocked`
Drain queue after freeze	R1	freeze	transfer 2 executed and no transfer `approved`
Payee change	R2	approval	transfer 1 executed with the new payee
Payment retry (timeout)	R3	retry	A's balance reduced by exactly 500
Plain transfer	control	approval	transfer 1 executed
Single payment	control	–	B's balance reduced by exactly 250
Freeze idle account	near-miss R1	–	A frozen and B's transfer executed
Edit before approval	near-miss R2	approval	transfer 1 executed with the corrected amount
Distinct payments	near-miss R3	–	A's balance reduced by exactly 300